SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA51 – Cryptography and Network Security

ASSIGNMENT 2 – CO2

Evaluate, Design and Defend a Secure Public-Key and Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network

CO2: Analyze public-key cryptosystem strategies using RSA and key exchange for encryption algorithms to strengthen information security.

Student Name	Register Number	Department / Section	Academic Year / Semester	Faculty Name	Nature	Duration	Total Marks	Date of Submission	Date of Evaluation
RAKSHITHA R	192324136	B.Tech AIDS	4th year	Dr.R.Dharani	Individual	1 week	50	23.10.26	24.10.26

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max Marks	Marks Awarded	Remarks / Evidence Expected
1	Symmetric Cryptosystem Selection and Modes of Operation	7	7	Explain DES, 3DES, Blowfish and RC5 structure, block/key characteristics and rounds; compare block and stream ciphers; analyse ECB, CBC, CFB, OFB and CTR, including IV/nonce use, leakage, error propagation and parallelism. Implement/simulate at least three algorithms and provide pattern/IV demonstrations and performance results.
2	Number-Theory Engine and Prime Selection for RSA	6	6	Implement GCD, Extended Euclidean Algorithm, modular inverse and fast modular exponentiation. Explain Euler's totient and gcd(e, φ(n)) = 1; compare trial division, Fermat and Miller–Rabin using toy candidates, including Carmichael numbers; record operation counts and discuss prime density and larger RSA sizes.
3	RSA Encryption/Decryption, Weaknesses and Padding	7	7	Generate RSA keys and implement encryption/decryption; verify a hand-checkable example; test deterministic/repeated messages, small-exponent behaviour, boundary inputs and invalid parameters. Demonstrate toy factorisation attacks; explain weak key generation, modulus limitations and OAEP padding.
4	Diffie–Hellman, Authentication and Key Management	7	7	Derive and implement Diffie–Hellman; show private/public values and shared-key computation; test toy examples and measure brute-force discrete-log effort. Demonstrate unauthenticated MITM and explain signatures/certificates. Propose key generation, storage, rotation, revocation, recovery and session-key lifetime policies.
5	ECC and Public-Key Strategy Comparison	5	5	Explain elliptic-curve public-key mathematics and demonstrate toy ECC key generation/agreement. Compare RSA, DH and ECC by key size, computation, bandwidth and deployment; discuss secure randomness, parameter validation and private-key protection; defend a strategy for constrained devices.
6	Digital Signatures, Hybrid Encryption and Quantum Readiness	7	7	Design hash-then-sign and signature verification; compare RSA, DSA/DSS and ECDSA. Design authenticated hybrid encryption for large files; test tampering of files, signatures and session keys; compare direct RSA and hybrid operation/file-size costs. Explain QKD, quantum threats to RSA/ECC and migration readiness.
7	Integrated Design Challenge: Diagnose, Redesign and Validate	11	11	A: Diagnose a bottleneck/weakness using earlier evidence (2). B: State a measurable redesign objective (1). C: Design a revised cryptographic architecture/protocol and explain properties, reduced operations and trade-offs (3). D: Integrate quantum readiness (1). E: Predict, implement and validate the redesign using comparable evidence (2). F: Defend the final design, residual risks and deployment trade-offs (2).
	TOTAL	**50**	50	**Marks to be awarded by the faculty.**

OVERALL REMARKS

Faculty feedback: Assignment Done Well. All Concepts were presented with an example

Student Signature	Faculty Signature	Program Director
rakshitha		

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present P = partly present ✗ = missing. Mark the status after checking the student's submitted report and supporting evidence.

Question	Evidence Expected	Status	Note
1	Algorithm/cipher comparison, mode comparison; three-algorithm test evidence; ECB pattern and IV/nonce demonstration; performance table.	☑	Evidence should be traceable to the submitted report, code or experimental output.
2	Mathematical derivations and pseudocode; designed test cases; modular operation counts; primality-test results including Carmichael numbers; engineering decision.	☑	Evidence should be traceable to the submitted report, code or experimental output.
3	RSA implementation and hand-worked example; prediction-versus-result table; attack/factorisation traces; padding analysis; security conclusions.	☑	Evidence should be traceable to the submitted report, code or experimental output.
4	DH derivation and message-sequence diagram; implementation output; MITM trace; authentication explanation; growth measurements; key-management policy.	☑	Evidence should be traceable to the submitted report, code or experimental output.
5	Toy ECC calculation; RSA/DH/ECC comparison table; constrained-device recommendation supported by Q2–Q4 evidence.	☑	Evidence should be traceable to the submitted report, code or experimental output.
6	Signature flow; hybrid-encryption design; tamper-test results; direct-RSA/hybrid comparison; quantum-readiness explanation; final architecture.	☑	Evidence should be traceable to the submitted report, code or experimental output.
7	Diagnosis, measurable objective, revised architecture/protocol, prediction, comparable experimental evidence, validation and engineering defence.	☑	Evidence should be traceable to the submitted report, code or experimental output.

IMPLEMENTATION EVIDENCE CHAIN(Tasks 5–7)

Evidence Item	Status	Observation
Source code	☑	Readable student-written core algorithms; language, dependencies and execution instructions stated.
Test inputs and keys	☑	Student-created plaintexts, keys, toy parameters, edge cases and test data recorded.
Expected behaviour	☑	Predicted ciphertexts, shared keys, signatures, attack outcomes and recovered plaintext documented.
Actual output	☑	Program output, screenshots or logs included and reproducible.
Intermediate calculations	☑	Modular arithmetic, key-generation steps, protocol values and transformations shown.
Attack evidence	☑	Toy-only attack traces for MITM, factoring, weak parameters, pattern leakage or tampering where required.
Performance measurements	☑	Operation counts, loop/division counts, execution time and input/key-size conditions recorded.
Boundary/failure tests	☑	Invalid parameters, boundary messages, repeated inputs, tampering and relevant failure cases considered.
Interpretation	☑	Results connected to mathematical properties, security assumptions and deployment constraints.
Consistency	☑	Code, raw data, tables, graphs, diagrams and report claims agree.

SUBMISSION CHECKLIST (not separately marked)

Requirement	Status
All questions Q1–Q7 and Q7 parts A–F answered	☑ Yes ☐ Partly ☐ No
All required mathematical derivations and pseudocode included	☑ Yes ☐ Partly ☐ No
Core algorithms implemented by the student; libraries used only for cross-checking	☑ Yes ☐ Partly ☐ No
Student-created toy parameters and test data used for all attack demonstrations	☑ Yes ☐ Partly ☐ No
Raw experimental data, operation counts and execution-time conditions provided	☑ Yes ☐ Partly ☐ No
Tables/graphs and protocol or message-sequence diagrams included	☑ Yes ☐ Partly ☐ No
Predictions compared with actual experimental evidence	☑ Yes ☐ Partly ☐ No
Security limitations, trade-offs and deployment decisions justified	☑ Yes ☐ Partly ☐ No
Source code and reproducible execution procedure provided	☑ Yes ☐ Partly ☐ No
Report numbering, cross-references and figures are consistent	☑ Yes ☐ Partly ☐ No
No unfilled template placeholders remain	☑ Yes ☐ Partly ☐ No
Student can explain and defend the work during verification/viva	☑ Yes ☐ Partly ☐ No

Safety note: All attack demonstrations must use only student-created toy parameters and test data. Do not use classroom code to protect real patient data.

Assessment note: A brief verification/viva may assess algorithmic and protocol design, source code, test-data/key-set design, predictions, experimental results, security analysis and the Q7 redesign.